Proyecto:
Arbolado Urbano

Integrantes:

Maria Fernanda Hernandez Ventura
Porfirio Omar Acevedo Benitez
Daniel Dominguez Sanchez
Brandon Alberto Jimenez Gil

Reporte técnico — Arbolado urbano

1. Análisis del problema

En este proyecto hicimos un sistema para manejar el arbolado urbano de Nueva York con el Censo de Árboles 2015 (666,134 registros). Como es mucha información, trabajamos con las 10 especies que más aparecen y máximo 8,000 árboles por especie (hasta 80,000 registros; `main.py` fija `LIMITE_ESPECIES = 10` y `ARBOLES_POR_ESPECIE = 8000`). De cada árbol guardamos `tree_id`, especie, diámetro (`tree_dbh`), salud, estado y barrio. El CSV solo se usa para la carga inicial, con el módulo `csv`. Las estructuras las programamos nosotros, con nodos y apuntador al padre `p`.

Problema	Necesidad	Estructura	Operación en el código
1	Localizar por `tree_id` y listar especies en orden alfabético sin ordenar	ABB por `tree_id` y ABB por especie	`iterative_tree_search`, `inorder_tree_walk`
2	Dar de baja un registro conservando el orden	ABB	`tree_delete` con `transplant`
3	Costo logarítmico aunque los `tree_id` lleguen crecientes	AVL por `tree_id`	`avl_insert`, `avl_delete`, rotaciones
4	Rango de diámetros, mínimo, máximo y sucesor	AVL por diámetro	`buscar_rango_avl`, `tree_minimum`, `tree_maximum`, `tree_successor`

Respuesta a la pregunta guía: el ABB nos sirve para buscar y mantener el orden, pero su altura depende del orden en que llegan los datos. El AVL se balancea solo.

2. Especificación de los TDA

TDA Diccionario ordenado. Lo implementamos de dos formas: como ABB y como AVL. Cada nodo guarda la clave `key`, una lista `registros` con los árboles que tienen esa clave, los hijos `left` y `right`, el padre `p` y, en el AVL, la altura `h` (una hoja vale 0 y el árbol vacío -1). El árbol guarda la raíz `root`, el total de nodos, el total de registros y, en el AVL, el número de rotaciones. Invariante del ABB: el recorrido inorden queda en orden creciente. Invariante del AVL: además de ser ABB, la diferencia de alturas entre el hijo izquierdo y el derecho es como máximo 1 en todos los nodos.

Operación	Precondición	Poscondición	Complejidad ABB	Complejidad AVL
`insert(T, k, reg)`	ninguna	`reg` queda en el nodo de clave `k` (nodo nuevo si no existía); invariante se conserva	O(h), O(n) peor caso	O(log n)

Operación	Precondición	Poscondición	Complejidad ABB	Complejidad AVL
`search(T, k)`	ninguna	devuelve el nodo con clave `k` o `None`	O(h)	O(log n)
`minimum / maximum`	árbol no vacío	nodo de menor / mayor clave	O(h)	O(log n)
`successor(x)`	`x` pertenece al árbol	nodo con la menor clave mayor que `x.key` o `None`	O(h)	O(log n)
`delete(T, z)`	`z` pertenece al árbol	`z` ya no está; orden conservado	O(h)	O(log n)
`inorder / preorder / postorder`	ninguna	lista de nodos en ese orden	Θ(n)	Θ(n)
`range(T, a, b)`	a ≤ b	todos los registros con clave en [a, b], en orden	—	O(log n + m)
`check_invariant`	ninguna	`True`/`False` (el AVL también devuelve el nodo infractor)	Θ(n)	Θ(n)

Aquí h es la altura y m el número de registros devueltos por el rango.

3. Justificación de la clave y de los duplicados

Para cada problema usamos una clave diferente, así que al final tenemos cuatro índices que apuntan a los mismos registros:

Índice	Clave	Por qué esa clave
ABB	`tree_id`	Es única; permite búsqueda exacta y baja directa .
ABB	`especie`	Su inorden entrega las 10 especies en orden alfabético sin llamar a `sorted()`.
AVL	`tree_id`	El censo asigna ids crecientes; un ABB se degenera en lista (altura n − 1), el AVL conserva O(log n).
AVL	`diametro`	El orden numérico habilita rango [a, b], mínimo, máximo y sucesor.

Elegimos `tree_id` para buscar y dar de baja porque no se repite, pero con él solo no se puede saber qué árboles miden entre 10 y 20 pulgadas. Por eso también indexamos por especie y por diámetro.

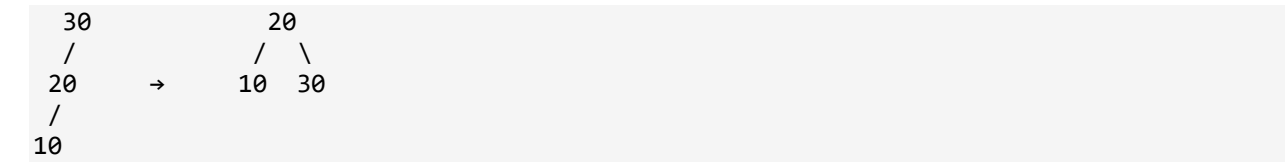
Claves duplicadas. Muchos árboles son de la misma especie o tienen el mismo diámetro, así que decidimos encadenarlas: si al insertar ya existe un nodo con esa clave, el registro se agrega a la lista `registros` de ese nodo y no se crea otro nodo. Así cada nodo tiene clave única, el orden estricto (<) se cumple y las rotaciones no tienen que preocuparse por empates. Usamos esta misma política en el ABB

y en el AVL. Al dar de baja por `tree_id`, si el nodo tiene más de un registro solo se quita ese registro, y si tiene uno se elimina el nodo. Por eso `total_nodos` cuenta claves distintas y `total_registros` cuenta árboles.

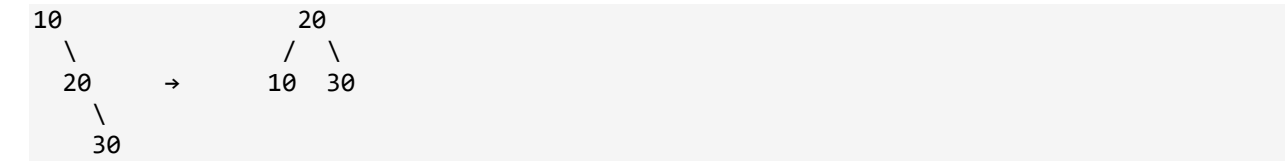
4. Las cuatro rotaciones

Después de cada inserción o eliminación, la función `rebalancear_hacia_arriba` sube desde el padre del nodo hasta la raíz. En cada nodo recalcula la altura `h` y el factor de balance `fb = h(izq) - h(der)`. Si `fb` llega a 2 o -2, se escoge la rotación según el `fb` del hijo más pesado. Cada rotación acomoda los apuntadores `p`, la raíz del árbol si hace falta y las alturas de los dos nodos que se mueven. Los cuatro ejemplos los corrimos con nuestro código y el verificador siempre dio `(True, 'OK')`.

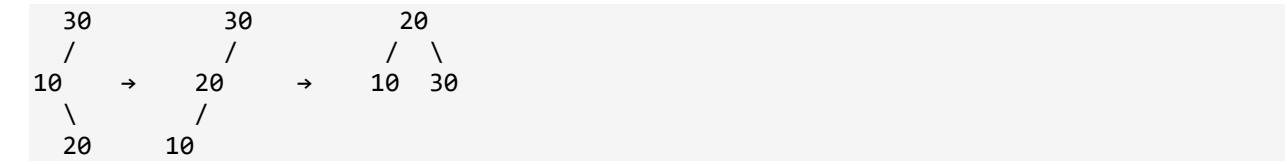
LL (rotación simple a la derecha). Insertamos 30, 20, 10. El nodo 30 queda con fb = 2 y su hijo izquierdo (20) con fb = 1, entonces se hace `right_rotate(30)`: el 20 sube, el 30 baja como hijo derecho y el subárbol derecho que tenía el 20 (aquí vacío) pasa a ser hijo izquierdo del 30.



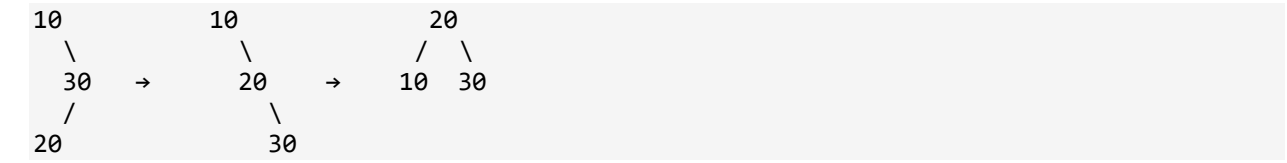
RR (rotación simple a la izquierda). Insertamos 10, 20, 30. El 10 queda con fb = -2 y su hijo derecho (20) con fb = -1, así que se hace `left_rotate(10)`. Queda el mismo resultado: 20 de raíz con hijos 10 y 30.



LR (izquierda-derecha). Insertamos 30, 10, 20. El 30 tiene fb = 2, pero su hijo izquierdo (10) tiene fb = -1, o sea que está cargado hacia la derecha. Por eso primero se hace `left_rotate(10)` y luego `right_rotate(30)`; el contador de rotaciones sube 2.



RL (derecha-izquierda). Insertamos 10, 30, 20. El 10 tiene fb = -2 y su hijo derecho (30) tiene fb = 1, así que primero se hace `right_rotate(30)` y luego `left_rotate(10)`.



En los cuatro casos se llega al mismo árbol (preorden 20, 10, 30). También probamos casos con subárboles: insertar 50, 30, 70, 20, 40, 45 (LR en el 50) deja el preorden 40, 30, 20, 50, 45, 70, y insertar 50, 30, 70, 60, 80, 65 (RL en el 50) deja 60, 50, 30, 70, 65, 80. Al eliminar se usa `fb(hijo) >= 0` (o `<= 0`) para decidir la rotación simple, porque después de borrar el hijo pesado puede tener fb = 0.

5. Resultados

Probamos el ABB contra el AVL insertando las claves 1..n (n = 1,000, 2,000, 4,000 y 8,000) con las funciones de `operaciones.py`, una vez en orden ascendente y otra en orden aleatorio (semilla 42). Para la búsqueda medimos el promedio de 1,000 búsquedas de claves que sí existen (en µs) y para la construcción el tiempo de las n inserciones (en s). Cada celda es una sola corrida en la misma computadora, por lo que los tiempos son aproximados, pero las alturas son exactas.

n	Orden	Altura ABB	Búsqueda ABB (µs)	Construcción ABB (s)	Altura AVL	Búsqueda AVL (µs)	Construcción AVL (s)	Rotaciones AVL	$\lfloor \log_2 n \rfloor$	$1.44 \cdot \log_2(n+2)$
1,000	ordenada	999	25.13	0.027	9	0.63	0.006	990	9	14.4
1,000	aleatoria	20	0.82	0.002	11	0.65	0.006	689	9	14.4
2,000	ordenada	1,999	45.45	0.113	10	0.46	0.007	1,989	10	15.8
2,000	aleatoria	23	0.67	0.003	12	0.87	0.009	1,407	10	15.8
4,000	ordenada	3,999	75.83	0.345	11	0.76	0.015	3,988	11	17.2
4,000	aleatoria	25	0.67	0.007	13	0.55	0.016	2,719	11	17.2
8,000	ordenada	7,999	162.59	1.426	12	0.77	0.033	7,987	12	18.7
8,000	aleatoria	35	0.93	0.014	15	0.71	0.037	5,436	12	18.7

Lo que observamos: (1) con las claves ordenadas el ABB se vuelve una lista: la altura es n - 1 y el tiempo de búsqueda crece casi proporcional a n (de 25 a 163 µs entre 1,000 y 8,000), mientras que el AVL se queda entre 0.5 y 0.8 µs. (2) Construir el ABB con datos ordenados crece de forma cuadrática (de 0.027 s a 1.426 s, unas 53 veces más con 8 veces más datos) y el AVL casi lineal (de 0.006 s a 0.033 s). (3) Las alturas del AVL siempre quedaron por debajo de la cota teórica $1.44 \cdot \log_2(n+2)$. (4) Con orden aleatorio el ABB se porta bien y hasta construye más rápido porque no hace rotaciones, pero su altura (20 a 35) es mayor que la del AVL (11 a 15) y no está protegido si los datos llegan ordenados. El verificador del AVL dio `True` en las ocho corridas.

Resultados con el subconjunto asignado. Al correr `main.py` con `export.csv` se cargaron 80,000 árboles de 10 especies, 8,000 de cada una: Norway maple, London planetree, Sophora, pin oak, littleleaf linden,

cherry, honeylocust, ginkgo, Japanese zelkova y Callery pear. Construir los dos ABB tardó 0.4987 s y los dos AVL 0.7553 s. Estos son los datos de la opción 6 del menú:

Estructura	Nodos	Registros	Altura	Invariante	Rotaciones
ABB por especie	10	80,000	6	True	—
ABB por tree_id	80,000	80,000	200	True	—
AVL por tree_id	80,000	80,000	19	True	69,859
AVL por diámetro	71	80,000	6	True	51

Con 80,000 nodos, $\lfloor \log_2 n \rfloor = 16$ y la cota $1.44 \cdot \log_2(n+2)$ es 23.4, así que la altura 19 del AVL por tree_id está dentro de lo esperado. El ABB por tree_id llegó a altura 200 y no a 79,999 porque en este subconjunto los ids no vienen estrictamente ordenados; aun así es más de 10 veces la altura del AVL. El AVL por diámetro solo tiene 71 nodos porque hay 71 diámetros distintos y los demás árboles quedan encadenados en la lista `registros` de cada nodo, por eso su altura es 6 y casi no rota. En el ABB de especies el preorden empieza con pin oak como raíz, y en el AVL de diámetros la raíz es 38.0 y el inorden empieza en 0.0, 1.0, 2.0, ... en orden creciente.

Una observación: el inorden de las especies queda así: Callery pear, Japanese zelkova, London planetree, Norway maple, Sophora, cherry, ginkgo, honeylocust, littleleaf linden, pin oak. Las que empiezan con mayúscula salen antes que las de minúscula porque Python compara por código de carácter, no por alfabeto. El orden es correcto para el ABB, pero para un orden alfabético que ignore mayúsculas habría que usar `especie.lower()` como clave. Los valores de control del docente no los tenemos aquí, por lo que falta compararlos con estos números.

6. Análisis de complejidad

Casi todas las operaciones recorren una sola rama del árbol, por eso su costo es O(h), con h la altura. En el ABB h puede ir de $\log_2 n$ (caso bueno) hasta n - 1 (claves ordenadas, como vimos en la sección 5). En el AVL $h \leq 1.44 \cdot \log_2(n + 2)$, porque un AVL de altura h tiene por lo menos F(h+3) - 1 nodos (F = Fibonacci), que crece como ϕ^h.

Operación (función)	ABB	AVL	Justificación
Búsqueda (`iterative_tree_search`)	O(h) ≤ O(n)	O(log n)	Un descenso por rama, sin recursión.
Inserción (`tree_insert`, `avl_insert`)	O(h)	O(log n)	Descenso O(h); el AVL sube por la rama recalculando `h` y aplica a lo más una rotación (simple o doble) por inserción.
Eliminación (`tree_delete`, `avl_delete`)	O(h)	O(log n)	`tree_minimum` del subárbol derecho y `transplant` son O(h); el AVL puede rotar en varios niveles al subir, a lo más O(log n) rotaciones.

Operación (función)	ABB	AVL	Justificación
Rotación (`left_rotate`, `right_rotate`)	—	O(1)	Reasigna un número constante de apuntadores y recalcula dos alturas.
Mínimo, máximo (`tree_minimum`, `tree_maximum`)	O(h)	O(log n)	Descenso por un solo lado.
Sucesor (`tree_successor`)	O(h)	O(log n)	Mínimo del hijo derecho o ascenso por `p`.
Recorridos (inorden, preorden, postorden)	Θ(n)	Θ(n)	Cada nodo se visita una vez; la pila de recursión llega a O(h).
Rango (`buscar_rango_avl`)	—	O(log n + m)	Solo se entra a `left` si `key > a` y a `right` si `key < b`; se visitan las dos ramas frontera más los m resultados.
Verificadores, altura por niveles	Θ(n)	Θ(n)	Recorren todos los nodos; sirven para pruebas, no para el flujo normal.
Baja completa (`main.py`, opción 3)	2 búsquedas + 2 bajas O(h)	2 búsquedas + 2 bajas O(log n)	Una baja por cada índice: ABB por id, ABB por especie, AVL por id, AVL por diámetro.

Espacio. Θ(n) nodos por índice. Cada árbol se referencia desde los cuatro índices, pero en los de especie y diámetro los duplicados se agrupan en un solo nodo, así que hay menos nodos que registros. Las operaciones iterativas usan O(1) de memoria extra y los recorridos recursivos O(h) por la pila; con claves ordenadas en un ABB esa profundidad puede llegar a n, por eso en `main.py` usamos `sys.setrecursionlimit(100000)`.

Rotaciones en la práctica. Con inserción ordenada se hicieron 990, 1,989, 3,988 y 7,987 rotaciones para n = 1,000, 2,000, 4,000 y 8,000, o sea casi una por inserción. Entonces rebalancear cuesta O(1) por inserción en promedio y lo que domina es el descenso por la rama, O(log n).

7. Pruebas

Probamos `operaciones.py` con los casos de la tabla y en cada uno revisamos el invariante con `verificar_invariante_abb` o `verificar_invariante_avl`. Todas las pruebas pasaron.

Prueba	Entrada	Resultado esperado y obtenido
Rotación LL	insertar 30, 20, 10	preorden 20, 10, 30; 1 rotación
Rotación RR	insertar 10, 20, 30	preorden 20, 10, 30; 1 rotación
Rotación LR	insertar 30, 10, 20	preorden 20, 10, 30; 2 rotaciones

Prueba	Entrada	Resultado esperado y obtenido
Rotación RL	insertar 10, 30, 20	preorden 20, 10, 30; 2 rotaciones
LR con subárboles	insertar 50, 30, 70, 20, 40, 45	preorden 40, 30, 20, 50, 45, 70
RL con subárboles	insertar 50, 30, 70, 60, 80, 65	preorden 60, 50, 30, 70, 65, 80
Eliminación ABB, hoja	borrar 20 de 50, 30, 70, 20, 40, 60, 80, 35	preorden 50, 30, 40, 35, 70, 60, 80
Eliminación ABB, un hijo	borrar 40 (hijo 45)	preorden 50, 30, 20, 45, 70, 60, 80
Eliminación ABB, dos hijos	borrar 30 (lo sustituye su sucesor 35)	preorden 50, 35, 20, 40, 70, 60, 80
Eliminación ABB, raíz con dos hijos	borrar 50 (lo sustituye 60)	preorden 60, 30, 20, 40, 35, 70, 80
Duplicados	insertar 5, 5, 5, 3	2 nodos, 4 registros, 3 encadenados en la clave 5
Extremos y sucesor	AVL con 1..7	mínimo 1, máximo 7, sucesor de 4 = 5
Rango	AVL con 1..7, [3, 5]	registros 3, 4, 5
Estrés AVL	600 inserciones y 300 bajas aleatorias, invariante tras cada operación	siempre `True`; 300 nodos en el árbol y 300 en el inorden
Degeneración	inserción ordenada de 1,000 a 8,000 claves	altura ABB $n - 1$, altura AVL $\lfloor \log_2 n \rfloor$ (sección 5)